G078 문희주

AI 서비스 Mini Project — 과제 2 (신규 AI Agent) 최종 보고서

WISH FINDER

자막 기반 영상 장면 찾기 AI Agent (첫 적용 대상: NCT WISH 영상 3편)

한 줄 요약 정확한 대사나 영상 제목은 기억나지 않지만 "어떤 장면이었는지"만 어렴풋이 기억하는 팬이, 한국어로 장면을 설명하면 영상의 정확한 재생 시점을 찾아 주는 근거 기반 검색 **Agent**이다. 자막에서 후보를 찾고, 원본 자막과 다시 대조해 검증한 뒤, 검증을 통과한 장면만 영상 링크와 함께 보여 준다. 중심 성과는 자막 근거와 영상 시점을 코드로 연결하고**,** 의미 판정과 최대 **2**번의 재검색을 결합해 브라우저에서 실제로 동작하는 서비스로 구현한 것이다.

결과 한눈에

- 동작: 브라우저에서 질문 입력 → 진행 단계 표시 → 검증된 장면 카드 → 해당 시점 영상 재생까지 실제로 동작한다.
- 정확도: 정답 평가셋의 장면 질문 18개 중 Agent가 "찾음"으로 응답하고 1위가 정답인 것이 **16**개**(88.9%)**이다. 같은 조건의 일반 검색(Hybrid)은 11~15개였다. 다만 차이가 1~5문항이고 질문당 1회만 실행해서, Agent가 더 낫다고 단정할 수는 없다. 지표의 집계 기준은 3.1절에 정의했다.
- 안전성(확인한 범위): 이 평가셋 20문항에서 근거 없이 화자를 단정한 경우 **0**건, 자막으로 알 수 없는 요청 2개 모두 거부, 화자 질문 4개 중 3개를 성공했다. 이는 평가셋 안의 결과이며, 검증을 통과했다는 것이 그 장면이 사용자 의도에 맞는다는 보장은 아니다.
- 개선 전/후**(v2 → v3)**: 1위 적중은 16/18로 같았다. 화자 질문 처리가 2/4 → 3/4로, LLM 호출이 16% 줄었다(50 → 42회). 반대로 입력 토큰이 24% 늘었고, "찾음"으로 응답한 사례 중 정답 비율이 16/16에서 **16/17(94.1%)**로 낮아졌다. 틀린 장면을 "찾음"으로 응답한 사례가 0건에서 1건으로 늘었기 때문이다.
- 추가 측정**(2026-10-22)**: 이중 첫 검색을 켜도 1위 적중은 16/18로 같고 검색 호출만 늘어 기본값은 꺼 두었다. 청크 길이는 20초/10초를 유지했다.
- 한계: Multi-Agent(여러 Agent의 협업)는 구현하지 않았다. 같은 의미의 질문도 표현에 따라 찾기도 하고 못 찾기도 하며, 후속 질문에서 틀린 장면을 내는 사례가 있다.

저장소: https://github.com/creepygrape/skala-AICapstone-mini-2 (실행 방법은 부록 B)

읽기 전에: 용어 설명

표 1. 이 보고서에서 쓰는 용어

용어	뜻
cue(자막 한 줄)	자막 파일(SRT)의 한 항목. 시작·종료 시각과 텍스트로 이루어진다. 이 서비스는 cue를 유일한 근거로 쓴다.
A / B / C	비교하는 세 방식. A = BM25 키워드 검색, B = Hybrid 검색(키워드 + 의미), C = Hybrid 검색 + Agent(검증·재검색).
v2 / v3	개선 전 / 개선 후 버전(v3: 화자 질문 판정, 번역 자막 대응, 중복 재검색 제거 등 적용).
결과 상태	찾음(found) / 못 찾음(not_found) / 애매함(ambiguous) / 자막으로 알 수 없는 요청(unsupported).
정답 평가셋(gold)	원본 cue를 검토해 정답 구간을 정하고 사용자가 20문항을 승인한 평가용 질문(장면 질문 18, 자막으로 알 수 없는 요청 2). 영상 화면과의 대조는 영상별 3점 표본으로만 했다(4.3절).
1위 적중 / 상위 3개 적중	반환한 1위 결과의 링크 시각이 정답 구간(±10초) 안인 문항 수 / 상위 3개 중 하나라도 그런 문항 수. Agent(C)는 "찾음"으로 응답한 경우만 센다(3.1절).

1. 진행 사항 소개

1.1 과제 주제와 문제 정의

사용자 문제. 팬은 "마니또를 뽑다가 자기 자신을 뽑은 장면"처럼 상황만 기억한다. 영상 제목이나 정확한 대사를 모르면 일반 키워드 검색으로는 찾기 어렵고, 1시간짜리 영상을 직접 돌려 보는 데도 시간이 든다.

해결 방식. 한국어로 장면을 설명하면 자막에서 후보를 찾고, 원본 자막과 대조해 검증한 뒤 영상 주소와 시작 시점을 알려 준다. 화면에는 해당 시점의 영상과 앞뒤 자막을 함께 보여 준다.

> 기술적 제약(사용자 문제와 구분). 실제 영상에는 한국어·일본어 화면 자막이 있지만, 확보할 수 있는 추출 자막은 영어 번역본이고 화자(말한 사람) 표시가 없다. 그래서 이 서비스는 음성과 영상을 이해하지 않고 자막 텍스트(대사와 괄호 속 화면 편집 설명)만 쓰며, 말한 사람은 단정하지 않는다. 예를 들어 "(RIKU chose RIKU)"는 화자 표시가 아니라 화면 편집 설명이다.

1.2 주요 task와 결과

표 2. 주요 task와 결과

영역	한 일	결과
데이터	자막 파일을 읽어 cue로 저장하고, 영상별로 20초 단위로 묶어 검색용 단위(청크)를 만들었다(영상 간 혼합 없음).	cue 3,750개 → 청크 869개, 영상 3편
검색(RAG)	키워드 검색(BM25)과 다국어 의미 검색(bge-m3)을 합친 Hybrid 검색을 만들었다.	A(BM25)·B(Hybrid) 기준선 구현
Agent	LangGraph로 분류 → 검색 → 검증 → (재검색) → 결과 정리 흐름을 구성했다.	재검색 최대 2번, 시간 제한, 오류 시 폴백
MCP	자막 조회 도구 4개를 MCP 서버로 만들고, 그래프가 실제로 호출한다.	읽기 전용 도구 4개
웹/API	FastAPI와 Vue로 대화형 화면을 만들고, 진행 단계를 실시간(SSE)으로 보여준다. 검색 기록, 앞뒤 자막, 후속 질문을 지원한다.	브라우저에서 end-to-end 동작
운영	LLM 백엔드를 환경변수로 교체(Ollama 개발 / OpenAI 제출)하고, LangSmith로 호출·토큰·지연을 기록한다.	LangSmith에서 요청별 토큰·지연 확인
평가·문서	A/B/C 평가 도구와 설계 결정 기록(ADR 11건)을 만들었다.	3장, 부록 C

1.3 대상 범위와 데이터 원칙

- 구현·데이터·평가는 **NCT WISH** 영상 **3**편으로 한정했다. 대상이 바뀌어도 코드를 고치지 않도록 대상별 정보(자막 언어, 번역 여부, 이름 사전)는 별도 설정 파일에 두었다. 다른 대상을 실제로 지원한다고 주장하지는 않는다.
- 제삼자의 원본 자막은 공개 저장소에 올리지 않는다. 화면에는 영상 링크와 짧은 인용만 보여 준다.
- 인용문**,** 시각**,** 영상 주소는 **LLM**이 쓰지 않고**,** 코드가 원본 **cue**에서 만든다**.** 자막에 이름이 나온 것(언급된 멤버)과 그 말을 한 사람(화자)은 구분한다.

2. 사용한 기술 요소

2.1 전체 구조

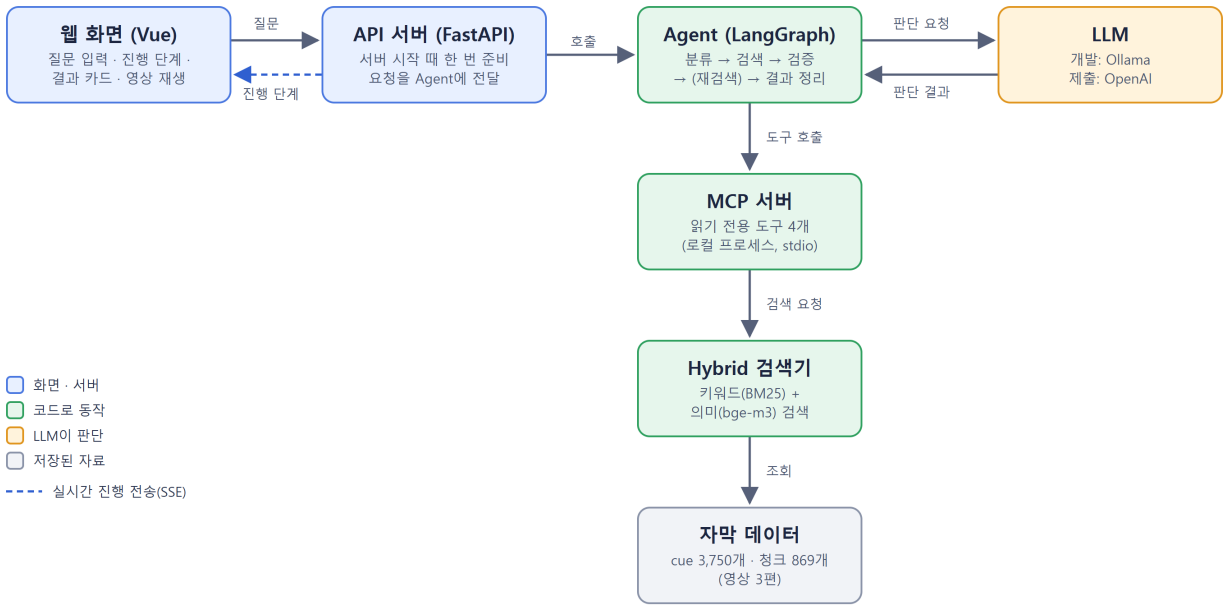
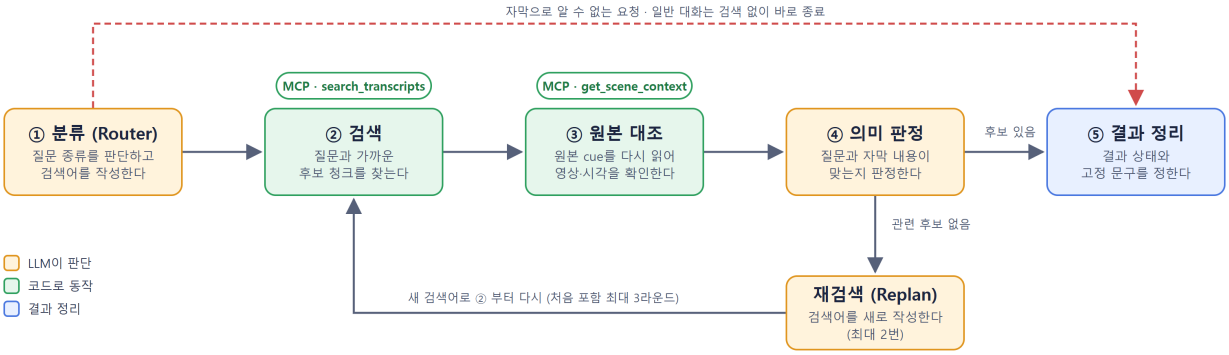


그림 1. 시스템 구성. 서버가 시작될 때 Agent, MCP 연결, 검색 인덱스를 한 번만 준비하고 요청마다 다시 만들지 않는다.

2.2 LangGraph Agent (단일 Agent 워크플로)



그래프 밖에서 쓰는 MCP 도구: get_video_metadata(서버 시작 때 영상 정보 읽기), get_cue_neighbors(화면의 앞뒤 자막 조회)
제한: 재검색 최대 2번 · LLM 호출 20초 · 전체 60초 · 남은 시간 25초 미만이거나 같은 검색어·새 후보 없음이면 재검색하지 않는다

그림 2. LangGraph 실행 흐름. 다음 단계는 코드의 조건문이 정하고, LLM은 세 곳(분류, 판정, 검색어 재작성)에서만 판단한다. 검색(②)은 search_transcripts, 원본 대조(③)는 get_scene_context를 호출한다.

표 **3. Agent** 단계별 역할

단계	하는 일
Router	질문을 분류(장면 검색 / 화자 질문 / 자막으로 알 수 없는 요청 / 일반 대화)하고, 자막 언어(대상 설정에서 정하며 지금은 영어)로 검색어를 만든다. 검색이 필요 없으면 바로 종료한다.
Search, Context	후보를 검색하고, 후보마다 원본 cue를 다시 읽어 영상, 시각, cue 포함 여부를 코드로 검증한다. 검증에 실패한 후보는 버린다.
Verification	원래 한국어 질문과 원본 자막(앞뒤 문맥 포함)을 비교해 관련 있음 / 불확실 / 관련 없음으로 판정한다.
Replan	관련 후보가 없고 남은 시간이 25초 이상일 때만, 핵심 명사를 바꿔 검색어를 다시 작성한다. 최대 2번(처음 검색 포함 3라운드)이며, 같은 검색어이거나 새 후보가 없으면 멈춘
Finalize	결과 상태와 고정 설명 문구를 정한다. 모델이 쓴 자유 문장은 사용자에게 보이지 않는다.

- 오류 처리: 모델이나 도구의 오류는 재검색 사유로 쓰지 않는다. 여기서 "검증"은 두 가지다. 코드의 원본 대조(cue 존재, 영상·시각·인용의 일관성)는 통과했지만 LLM 의미 판정을 마치지 못한 후보만 있으면 "부분 결과"로 응답하고 의미 판정을 건너뛰었다는 경고를 붙인다. 이전에 "불확실" 판정이 있었으면 "애매함"으로 응답하고, 아무것도 없으면 안전한 오류 코드만 보낸다. 실시간(SSE) 전송 중에는 진행 단계 이벤트만 보내고, 결과는 이 최종 응답 하나로만 보낸다. 즉 "검증 전 결과는 보내지 않는다"는 것은 원본 대조를 거치지 않은 후보를 중간에 화면에 보내지 않는다는 뜻이다.

2.3 MCP (Model Context Protocol)

표 **4. Agent**가 쓰는 **MCP** 도구 **2**개 (모두 읽기 전용, 파일 경로를 인자로 받지 않음)

도구	하는 일	쓰이는 곳
search_transcripts	질문으로 후보 청크를 검색	검색 단계(질문마다). 계산이 무거워 동시에 1건만 실행
get_scene_context	영상의 특정 시각 주변 원본 **cue** 조회	원본 대조 단계(후보마다). 영상 경계 안에서만 반환

표 **5. MCP** 서버가 제공하지만 **Agent**가 아니라 **API** 서버(화면)가 호출하는 도구 **2**개

도구	하는 일	쓰이는 곳
get_video_metadata	영상 제목·주소·확인 상태 조회	API 서버가 시작할 때 한 번 호출해 메모리에 저장
get_cue_neighbors	결과 cue의 앞뒤 cue 조회	화면의 앞뒤 자막(사용자가 시점을 선택할 때). API 서버가 호출

- 공식 Python SDK 2.x의 MCPServer를 쓰고, 로컬 프로세스(stdio)로 실행한다. 연결은 서버 시작 때 한 번 열어 모든 요청이 공유한다.

- 설계 선택과 대가: 자막 데이터는 MCP 서버만 읽는 단일 소유자 구조라서 API 서버도 영상 정보와 앞뒤 자막을 MCP를 거쳐 조회한다. 그 대가로 Agent용이 아닌 화면용 조회가 MCP 도구 계약에 섞인다.
- 도구를 나눈 이유: ① 입력이 다르다(질문 / 영상과 시각 / cue ID). ② 무게가 다르다. 검색만 계산이 무거워 동시에 1건으로 제한하고, 나머지는 가벼워 검색 중에도 호출할 수 있다. ③ 검색 결과의 청크 텍스트는 검색용으로 가공(중복 제거, 이어 붙이기)한 것이라, 인용과 시각은 get_scene_context가 돌려주는 가공하지 않은 원본 **cue**로 다시 확인한다. 청크 만들기에서 생긴 오류가 인용과 시각으로 번지지 않게 하기 위해서이다. ④ 파일 경로를 인자로 받지 않아 LLM이나 코드가 파일을 직접 다루지 못하게 하기 위해서이다. 앞뒤 자막 조회(get_cue_neighbors)는 화면을 만들면서 나중에 추가했고, 이미 쓰고 있는 도구의 입출력을 바꾸지 않으려고 따로 두었다.

2.4 RAG: Hybrid 검색과 원본 자막 근거

- 청킹: 영상별로 20초 단위를 10초씩 겹쳐 만든다. 영상을 가로질러 묶지 않으며, 각 청크는 포함된 cue 목록을 가지고 있어 결과를 항상 원본 cue로 되돌릴 수 있다. 자막 한 줄은 1~3초로 짧아 한 줄만으로는 의미 검색이 어렵고, 한 장면은 보통 수십 초에 걸쳐 이어진다고 보아 20초로 묶었다. 10초(절반)씩 겹치는 것은 장면이 단위의 경계에 걸리는 일을 줄이기 위해서이다. 10초 이하의 장면은 어느 한 단위에 온전히 들어가지만, 그보다 긴 장면은 한 청크에 다 들어간다고 보장하지 못한다. 이 값은 설계 때 정한 가정이었고**, 10/5·30/15·40/20**초와 비교한 결과 **20**초**/10**초를 유지했다**(**부록 **E).**
- 검색: 키워드 검색(BM25) 상위 20개와 의미 검색(bge-m3, 다국어) 상위 20개를 RRF로 합친다. RRF는 각 목록에서 순위가 r인 후보에 **1 / (60 + r)** 점수를 주고 두 목록의 점수를 더하는 방식이며, 60은 기본값을 사용했다. 합친 뒤 같은 영상에서 cue가 겹치거나 시작이 10초 이내인 후보는 하나만 남기고, 위에서부터 최대 **3**개(top_k=3)를 후보로 쓴다(BM25는 질문과 단어가 하나라도 겹치는 청크만 순위에 오른다).
- 대표 근거 **cue**: 후보 청크 안의 cue마다 의미 검색 임베딩(bge-m3, 길이 1로 정규화)과 그 검색에 쓴 질문의 임베딩의 내적을 구해 가장 높은 cue 한 개를 고른다(연속 구간이 아니라 단일 cue이며, 동점이면 청크 안에서 먼저 나온 cue). 질문은 해당 검색에 실제로 넣은 문자열이다. 즉 Router가 만든 영어 검색어이거나, 이중 첫 검색의 한국어 원문이다. 키워드 전용 모드는 질문과 겹치는 단어 수가 가장 많은 cue를 고른다. 사용자에게 보이는 인용과 시각은 이 cue를 원본에서 다시 읽어 만든다.
- 후보 **3**개 제한의 영향: 후보를 3개로 두면 판정 LLM의 토큰과 외부 전송량이 늘지 않는 대신, 정답 청크가 4위 이하이면 판정할 기회 자체가 없다. 실제로 이런 문항이 있었다(4.2절 실패 유형 점검).
- 질문 언어와 자막 언어가 다를 때: 이 프로젝트는 한국어 질문에 영어 번역 자막이다. Router가 핵심 명사의 동의어를 함께 넣어 자막 언어로 검색어를 작성하고, 판정 단계에서도 번역 때문에 단어가 달라도 뜻이 같으면 인정한다.
- 이중 첫 검색(선택 기능**,** 기본 꺼짐): 첫 검색에서 Router가 자막 언어로 작성한 검색어와, 사용자가 입력한 질문 그대로(입력한 언어 그대로)를 함께 검색해 번역 어휘 차이를 보완한다(3.4절).

2.5 의존성 주입

- FinderAgent는 필요한 것(MCP 연결, 영상 정보, 모델, 도구 호출 함수, 로그 설정 등)을 모두 생성자로 받는다. 운영에서는 서버가 실제 연결과 모델을 넣고, 테스트에서는 가짜 모델과 가짜 연결로 바꿔 넣는다.
- **LLM** 교체: 환경변수 하나(WISH_LLM_BACKEND)로 Ollama(개발, 무료)와 OpenAI(시연·제출, 과금)를 바꾼다. 프롬프트와 출력 형식은 공통 부분에 한 벌만 두고 각 백엔드는 전송만 맡아서, 그래프·MCP·검색·테스트 코드는 바뀌지 않는다.
- 두 백엔드의 호환성 테스트는 가짜 HTTP 응답으로 하므로 과금되지 않는다.

2.6 LangSmith와 관측성

- 선택 기능: WISH_LANGSMITH_ENABLED=1과 LANGSMITH_API_KEY가 모두 있을 때만 켜진다. 켜면 그래프 단계와 **LLM** 호출마다 모델명, 입출력 토큰, 지연이 하나의 트레이스로 올라가 호출별 비용을 볼 수 있다.
- 질문과 후보 자막이 함께 올라가므로 개인 비공개 프로젝트에서만 쓴다. 키가 없어도 핵심 기능은 동작하며 같은 내용이 로컬 로그 파일(logs/agent.jsonl)에도 남는다.
- 평가를 두 번(v2, v3) 실행한 결과가 LangSmith 프로젝트 wish-finder-eval에 올라가 있다. 개요 화면(그림 3)은 트레이스 **160**개, 모델 호출 **188**회, 오류율 **0%**, 토큰 **135.5K**, 비용 **$0.30**을 보여 준다. 이 값은 로컬 기록과 일치한다(호출 98 + 90 = 188회, 토큰 61,066 + 74,438 = 135,504). 비용은 LangSmith가 모델명과 토큰으로 계산한 값이라 실제 청구액과 다를 수 있다.

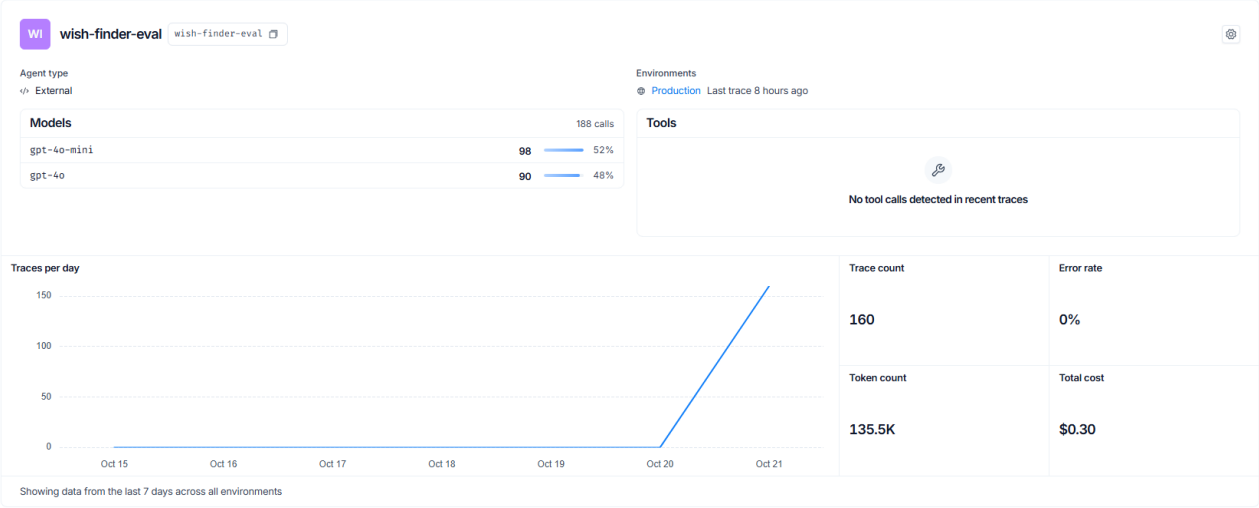


그림 3. LangSmith 프로젝트 wish-finder-eval 개요 (최근 7일, v2·v3 평가의 합계). 모델별 호출 수, 트레이스 160개, 오류율 0%, 토큰 135.5K, 비용 $0.30.

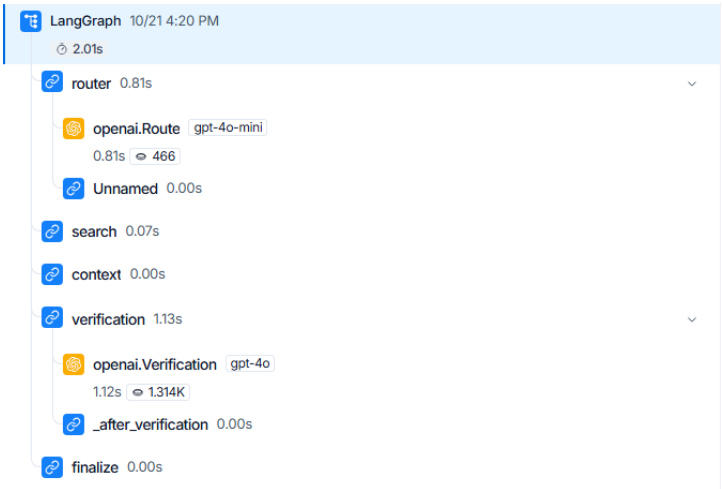


그림 4. LangSmith 트레이스 트리 (재검색 없이 끝난 평가 문항 1건, 질문은 비공개라 트리만 표시). 전체 2.01초 = router 0.81초(LLM, gpt-4o-mini, 466토큰) + search 0.07초 + context 0.00초 + verification 1.13초(LLM, gpt-4o, 1.314K토큰) + finalize. 단계 이름이 코드의 그래프와 그대로 대응한다.

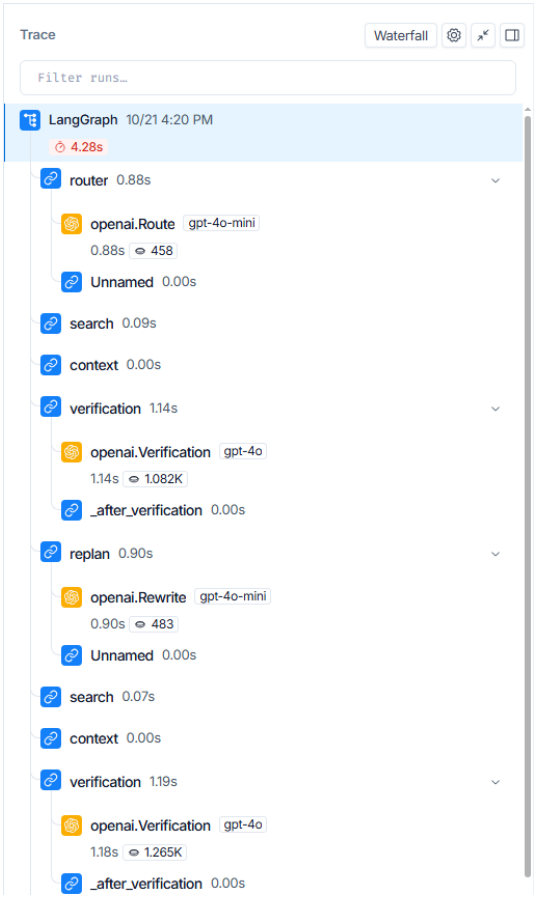


그림 5. LangSmith 트레이스 (재검색 1번, 2라운드). 전체 4.28초 = router 0.88초 + 1라운드(search 0.09초, verification 1.14초) + replan 0.90초(LLM, gpt-4o-mini, 483토큰) + 2라운드(search 0.07초, verification 1.19초). 맨 아래 finalize는 화면 밖으로 이어진다. 위 그림의 재검색 없는 건(2.01초, LLM 2회, 약 1.8K토큰)과 비교하면 재검색 1번이 지연, LLM 호출(2 → 4회), 토큰(약 3.3K)을 대략 두 배로 늘린다. 서로 다른 질문을 1번씩 본 것이라 통제된 비교는 아니다.

2.7 모듈화

표 **6.** 모듈 구성

디렉터리	책임
backend/app/main.py, sse.py	HTTP, 실시간 진행 전송(SSE), 서버 시작·종료 시 자원 관리, 오류를 사용자용 메시지로 변환
backend/app/agent/	LangGraph 그래프, LLM 공통 규칙과 Ollama·OpenAI 연결, 로그·LangSmith 기록
backend/app/mcp/	MCP 서버와 클라이언트, 동시 검색 제한
backend/app/retrieval/	자막 파서, 청크 생성, Hybrid 검색
backend/app/services/	도구 호출, 원본 대조 검증, Agent 없는 검색(A/B), 영상 정보
frontend/src/	Vue 화면, 실시간 이벤트 처리, YouTube 플레이어, 검색 기록 저장
eval/, scripts/	A/B/C 평가 도구(정답 평가셋 승인 확인 포함), 실행 스크립트

2.8 단위·통합 테스트

- 백엔드 **178**개 통과(macOS에서 2026-10-22 재실행. Windows에서는 권한이 없으면 건너뛰는 파일 링크 테스트 1개가 있어 177개 통과·1개 건너뜀), 프런트 **43**개 통과, 프런트 빌드 성공.
- 모델 호출은 가짜 모델과 가짜 HTTP 응답으로 대신해서 테스트가 과금되지 않고 결과가 항상 같다. 실제 MCP 서버(stdio)를 띄우는 테스트와, 합성 자막으로 MCP·LangGraph·API를 연결하는 통합 테스트도 포함한다.

표 **7.** 백엔드 테스트 영역별 내용 (합계 **178**개)

영역	검증 내용	개수
Agent 그래프	경로와 재검색 횟수 상한(0/1/2회), 같은 검색어·중복 후보에서 종료, 모델·도구 오류 시 폴백, 후속 질문과 이미 본 장면 제외, 이중 첫 검색	55
LLM 교체	Ollama/OpenAI 호환성(가짜 HTTP), LangSmith로 보내는 내용(가짜 세션)	19
자막·검색	원본 cue·시각·영상 경계, 괄호 속 편집 설명, 청크 만들기와 청크 길이 설정, 도구 호출과 근거 검증	37
설정·API·실시간 전송	대상 프로필, JSON API와 앞뒤 자막 조회, 실시간 이벤트의 순서·분할 수신·취소와 자원 정리	33
평가 도구	Agent 평가 경로, 정답 평가셋 승인 해시, 비공개 경계, 지표 계산	32
실제 MCP 통신	실제 stdio MCP 서버를 띄워 호출하고 종료	2

- 프런트 **43**개: 결과 카드(영상별 묶음, 선택 표시)·진행 단계·상세 패널 13개, 플레이어 수명과 정지 13개, 실시간 이벤트 분할 수신 7개, 진행 단계 계산 5개, 검색 기록 저장·복원 5개.
- 테스트가 다루지 못하는 것: LLM의 실제 응답 품질은 가짜 모델로 확인할 수 없어 3장의 평가로 확인한다. 화면의 클릭 동작은 자동 테스트가 아니라 Chrome에서 직접 확인했고, 브라우저 자동 점검 스크립트는 실행하지 않았다.

3. Before/After 실측 비교 (보너스 항목)

결론부터. 개선 후(v3)에도 1위 적중 수는 16/18로 같았다. 좋아진 것은 화자 질문 처리(2/4 → 3/4), LLM 호출 수(50 → 42회), 평균 지연(2.88 → 2.57초)이다. 나빠진 것은 입력 토큰(+24%)과 "찾음" 응답의 정확도다. 틀린 장면을 "찾음"으로 응답한 사례가 0건에서 1건으로 늘어 "찾음" 응답 중 정답 비율이 16/16에서 16/17(94.1%)이 됐다. 아래 모든 수치는 질문당 **1**회 실행한 값이므로 1~2문항 차이는 우연일 수 있다.

3.1 측정 조건

- 같은 입력: 정답 평가셋(gold) 20문항. 장면 질문 18개(일반 13, 같은 말을 하는 장면이 여러 개인 질문 1, 화자 질문 4)와 자막으로 알 수 없는 요청 2개. 같은 영상 3편, 같은 정답. 정답 구간은 어시스턴트가 원본 cue를 검토해 제안하고 사용자가 20문항을 승인한 것이며, 영상 화면과의 대조는 영상별 3점 표본에 한정한다(4.3절). 성공 기준은 아래 지표 정의를 따른다.

- **LLM**: OpenAI. 분류와 검색어 재작성은 gpt-4o-mini, 판정은 gpt-4o(temperature 0, seed 42). 시연과 제출은 OpenAI 기준이라 이 장의 수치는 모두 OpenAI로 측정한 값이다.
- 개선 전**(v2) /** 후**(v3)**: v3는 v2에서 드러난 문제 네 가지를 고친 버전이다. 같은 평가셋과 같은 명령으로 다시 측정했다.
- 고친 점 ① 화자 질문의 판정 기준: "누가 말했어?"라고 물어도 자막에는 말한 사람이 적혀 있지 않다. v2는 이 때문에 장면이 맞아도 "불확실"로 판정하는 일이 있었다. v3는 장면이 맞으면 "관련 있음"으로 판정하고, 말한 사람은 "확인 불가"로 따로 안내한다.
- 고친 점 ② 번역 자막 대응: 자막이 영어 번역본이라 한국어 질문의 단어와 자막의 단어가 다를 수 있다. 번역 때문에 단어가 달라도 뜻이 같으면 인정하도록 판정 지침을 바꾸고, 검색어에는 핵심 명사의 동의어를 함께 넣게 했다. 자막에 자주 나오는 멤버 이름의 표기는 설정 파일의 이름 사전으로 알려 준다.
- 고친 점 ③ 중복 재검색 제거: 재검색이 이미 판정한 후보를 다시 판정하거나 같은 검색어를 반복해 시간과 LLM 호출만 쓰는 일이 있었다. v3는 이미 판정한 후보는 다시 묻지 않고, 새 후보가 없으면 재검색을 멈춘다.
- 고친 점 ④ 거절된 후보를 보여 주며 검색어 재작성: 재검색에서 검색어를 다시 작성할 때 직전에 "관련 없음"으로 판정된 후보의 인용문을 함께 보여 준다. 같은 계열의 말을 또 찾지 않고 핵심 명사를 바꾸게 하기 위해서이다.

평가 지표 정의

표 **8.** 평가 지표 정의와 집계 기준

지표	분모	성공 기준 (평가 코드 **eval/metrics.py** 기준)
장면 1위 적중	정답 장면이 있는 18문항(화자 질문 4개 포함)	반환한 1위 결과의 영상이 정답이고, 링크 시각 floor(start_seconds) 한 점이 정답 구간 ±10초 안이다. 반환 구간이 길게 겹치는 것은 성공이 아니다. **Agent(C)**는 결과 상태가 "찾음"**(found)**일 때만 성공으로 센다**.** "애매함"·"부분 결과"로 반환한 후보가 정답이어도 성공에는 넣지 않고 원시 후보 적중으로 따로 기록한다. A·B는 항상 후보 목록(상태 "부분 결과")이라 1위 후보로 판정한다.
상위 3개 적중	위와 같음	반환한 상위 3개 중 하나라도 위 기준을 만족한다. C는 위와 같이 "찾음"일 때만 센다.
"찾음" 응답 정밀도	C가 "찾음"으로 응답한 문항	그중 1위가 정답인 비율(분자 = 1위 적중, 분모 = "찾음" 응답 수, 둘의 차이 = 틀린 장면을 "찾음"으로 응답한 수).
화자 질문 처리 성공	화자 질문 4문항	Router가 화자 질문으로 분류하고, 관련 장면이 1위 적중이며, "자막만으로 화자를 확인할 수 없습니다" 안내가 있고, 화자 단정이 없으며 오류가 아니어야 한다. 네 조건을 모두 만족해야 성공이다.
지원 범위 밖 요청 거부	시각·청각 전용 요청 2문항	결과 상태가 "알 수 없는 요청"(unsupported)이고 결과가 없으며 Router도 같은 의도로 분류했다.
근거 없는 화자 단정	전체 응답	화자 필드가 채워졌거나 사용자에게 보이는 설명문이 발화자를 주장한 건수. 인용문 안의 호명과 화면 편집 설명은 세지 않는다.

오류가 난 문항은 분모에 그대로 남기며, 정답이 없는 문항(못 찾음이 정답인 질문)은 확정된 것이 없어 이 정확도를 측정하지 못했다. v2에서는 화자 질문 1건이 "애매함"으로 끝났는데 그 후보의 1위는 정답이었다. 이 문항은 위 기준에 따라 v2의 1위 적중에 넣지 않았다(원시 후보 적중은 17/18).

첫 검색어 조건**.** 방식마다 처음 쓰는 검색어가 달라서(A·B는 질문 원문, C는 Agent가 영어로 작성) 그대로 비교하면 "검색어 차이"와 "Agent 효과"가 섞인다. 그래서 세 방식이 같은 첫 검색어로 시작하는 두 조건을 모두 측정했다.

표 **9.** 첫 검색어 비교 조건 (재검색 검색어는 **Agent**만 새로 작성한다)

조건	세 방식**(A·B·C)**이 모두 첫 검색어로 쓰는 것	이 장에서의 표기
조건 ① 질문 원문	사용자가 입력한 한국어 질문 그대로	Agent는 C-ko
조건 ② 영어 검색어	Agent(Router)가 질문을 보고 자막 언어(영어)로 작성한 검색어	Agent는 C

3.2 개선 전/후 (v2 → v3)

표 **10.** 개선 전/후 비교 **(OpenAI,** 같은 평가셋·같은 모델**,** 질문당 **1**회 실행**)**

지표 (괄호는 전체 문항 수)	개선 전 **(v2)**	개선 후 **(v3)**	변화
Agent 1위 적중 수 (장면 질문 18개)	16	16	같음
Agent 상위 3개 적중 수 (18개)	16	16	같음
Agent 결과 상태 (20개)	찾음 16, 애매함 1, 못 찾음 1, 알 수 없는 요청 2	찾음 17, 못 찾음 1, 알 수 없는 요청 2	애매함 1 → 찾음
화자 질문 처리 성공 수 (4개)	2	3	+1
자막으로 알 수 없는 요청 거부 수 (2개)	2	2	같음
틀린 장면을 "찾음"으로 답한 수	0	1	+1 (정답이 아닌 비슷한 내용의 다른 장면)
"찾음" 응답 정밀도	16/16 (100%)	16/17 (94.1%)	–5.9%p
B(Hybrid) 1위 적중 수, 조건 ② (18개)	13	15	+2
A(BM25) 1위 적중 수, 조건 ② (18개)	5	13	+8
Agent LLM 호출 수 (20문항 전체)	50	42	–16%
Agent 평균 / 최대 지연 (초)	2.88 / 6.26	2.57 / 6.74	평균 –11%
Agent 입력 / 출력 토큰	29,351 / 2,090	33,753 / 1,717	입력 +15%, 출력 –18%
C와 C-ko 합계 LLM 호출 / 토큰(입력·출력)	98 / 56,941·4,125	90 / 70,770·3,668	호출 –8%, 입력 **+24%**

- 재검색 횟수: 화자 질문 1건이 3라운드(LLM 6회)에서 1라운드(LLM 2회)로, 재검색으로 찾던 질문 2건이 2라운드에서 1라운드로 줄었다. 재검색 1번의 비용은 2.6절 그림 5에서 볼 수 있다(재검색 없는 건 2.01초·LLM 2회, 재검색 1번인 건 4.28초·LLM 4회).
- **1**위 적중 수는 같다**.** 위 표의 상태·적중 수·오답 수가 모두 맞는 설명은 이렇다. 한 문항을 얻었다: v2에서 "애매함"(1위는 정답이었으나 성공으로 세지 않음)이던 화자 질문이 v3에서 "찾음"이 되어 성공이 됐다. 한 문항을 잃었다: v2에서 "찾음"으로 정답이던 문항이 v3에서는 같은 "찾음"이지만 정답이 아닌 비슷한 내용의 다른 장면(영상 초반 약 3분대)을 반환했다. 그래서 "찾음"이 16건에서 17건으로, 정답 16건은 그대로, 오답 "찾음"이 0건에서 1건이 됐다. 이 오답이 라벨의 한계(실제로도 질문에 부합하는 다른 장면)인지 검색·판정 실패인지는 구분하지 못했다. 확인된 이득은 화자 질문 처리와 재검색 효율이고, 손해는 입력 토큰과 "찾음" 응답 정밀도다.
- A·B의 조건 ② 수치(A 5 → 13, B 13 → 15)는 두 실행에서 Router가 만든 검색어를 그대로 재사용한 결과다. 같은 시점에 Router가 핵심 명사의 동의어를 함께 넣도록 프롬프트를 바꿨으므로(고친 점 ②) 이것이 유력한 요인이지만, 네 가지를 한꺼번에 바꿨고 Router 검색어가 같은 설정에서도 실행마다 달라지므로(3.4절) 특정 변경의 단독 효과로 분리하지는 못했다**.** A·B에는 판정과 재검색이 없어 Agent의 그 부분과는 관계가 없다.

3.3 A / B / C 비교 (개선 후)

표 **11.** 1위 적중 수 (장면 질문 **18**개 중, 괄호는 상위 **3**개 적중 수)

첫 검색어 조건	**A** 키워드**(BM25)**	**B Hybrid**	**C Agent**
조건 ① 질문 원문	0	11 (상위 3개 13)	16 (C-ko)
조건 ② 영어 검색어	13 (상위 3개 15)	15 (상위 3개 16)	16 (상위 3개 16)

표 **12.** 지연과 호출 수 비교 (**v3** 평가 실행 기준, **OpenAI**)

방식	평균 지연	질문당 **Tool / LLM** 호출	비고
A 키워드(BM25)	0.002–0.005초	1.0–3.7 / 0	검색·원문 검증 단계만 측정. 이 단계는 LLM을 쓰지 않는다. 조건 ②의 검색어는 C의 Router(LLM)가 만든 것이며 그 생성 시간·비용은 포함하지 않는다
B Hybrid	0.05–0.17초	4.0 / 0	위와 같음. 조건 ① 0.17초, 조건 ② 0.05초(실행 순서에 따른 캐시 영향 가능)
C Agent (OpenAI, v3)	**2.57**초 (최대 6.74)	4.0 / 2.1	요청 전체(분류·검색·원문 대조·판정 LLM 호출 포함). LLM 호출 42회, 입력 33,753·출력 1,717 토큰

비교의 한계. A·B의 지연은 검색·원문 검증 단계만 잰 값이라, C의 요청 전체 지연(분류·판정 LLM 호출 포함)과 직접 비교할 수 없다. 조건 ②에서는 A·B도 LLM이 만든 검색어를 쓰므로 "LLM 없이 동작한다"는 말은 검색 단계에만 해당한다. LLM의 도움 없는 A·B의 성능은 조건 ①(질문 원문)이다.

Agent(C)는 조건에 따라 B보다 1~5문항을 더 맞혔다. 이 평가셋에서 확인한 안전성은 다음 범위로 한정한다. 자막으로 알 수 없는 요청 2개를 모두 거부했고, 화자 질문 4개 중 3개에서 "확인 불가"를 안내하며 장면을 맞혔고, 근거 없이 화자를 단정한 경우는 0건이었다. 반면 "찾음" 응답 17건 중 1건은 틀린 장면이었다(정밀도 94.1%). 검증을 통과한 근거만 보여 주지만 그것이 장면이 맞다는 보장은 아니다(4.2절). 대신 지연과 LLM·Tool 호출이 늘어난다.

3.4 추가 개선 시도: 이중 첫 검색

왜 했나. 같은 의미의 질문이 표현에 따라 찾기도 하고 못 찾기도 했다(4.2절). 원인을 보니 Router가 만든 영어 검색어와 번역된 자막의 단어가 달랐다(예: 검색어는 "manitto", 자막은 "Secret Santa"). 반면 질문을 한국어 그대로 의미 검색에 넣으면 정답이 1위로 나왔다.

어떻게 했나. 첫 검색에서 Router의 영어 검색어와 질문 원문을 둘 다 검색하고, 두 결과를 순위대로 번갈아 채워(Router 결과 먼저) 판정할 후보 3개를 만든다. LLM 호출과 판정 후보 수는 늘지 않고 검색만 1번 늘어난다. 재검색 라운드, 후속 질문, 알 수 없는 요청에는 적용하지 않으며, 기본값은 꺼짐이다.

표 **13.** 이중 첫 검색 동작 확인 (평가셋이 아닌 직접 실행, 질문당 **1**회)

질문 (실제 자막, **OpenAI, 1**회 실행)	미적용 (기존)	적용 (이중 첫 검색)
마니또 "…자기 자신을 뽑은 장면"	못 찾음, 3라운드, 7.7초	찾음 **3:09, 1**라운드, **3.4**초
구운 오징어 / 4월 20일 앨범 / 한우 축제	찾음, 1라운드, 2.4 / 2.9 / 2.6초	같은 시점을 찾음, 4.3 / 4.6 / 4.5초

- 표의 질문은 점검용으로 고른 것이다: 마니또는 기존에 못 찾던 사례이고, 오징어·4월 20일·한우는 이미 찾던 질문이 계속 찾아지는지 보려고 골랐다. 질문을 체계적으로 선정한 것이 아니라서 정확도 판단은 아래 정답 평가셋 재측정으로 했다.
- 확인된 것: 못 찾던 마니또 질문 두 표현을 1라운드에 찾았다. 장면 질문 10건이 모두 "찾음"이었고, 그중 오징어(45:53), 료칸(2:18), 곰(58:04), 4월 20일(34:46), 한우(0:39)는 사용자가 확인한 시점과 일치했다. 김치찌개는 1위가 확인 시점(10:33)이 아닌 같은 영상의 다른 관련 장면(8:37)이었고, 적용하지 않은 실행에서도 같은 결과가 나온 적이 있어 이 기능 때문인지는 구분하지 못했다.
- 구현 중에 발견한 성능 저하: 코드 리뷰에서 "같은 청크에서 검색어마다 고른 근거 cue가 다른데 Router 쪽만 남긴다"는 지적을 받아 후보를 근거 cue 기준으로 합치게 고쳤다. 그런데 이때 점수를 합산해 순위를 매겼더니 실제 자막에서 오히려 정답이 밀려났다. 두 검색이 함께 고른 약한 후보가 1위가 되어 후보 3자리 중 하나를 차지하고, Router의 2위였던 정답이 밀려난 것이다("구운 오징어" 질문이 적용 전에는 4/4번 45:53, 수정 직후에는 3/3번 42:41). LLM 없이 같은 검색 결과로 원인을 확인한 뒤 점수 합산 대신 순위를 번갈아 채우는 방식으로 바꿨고, 이 경우를 테스트로 고정했다. 단위 테스트만으로는 이 문제를 찾지 못했고 실제 자막으로 확인해야 했다.
- 단점: 검색이 두 번이라 지연이 늘 것으로 예상했고, 위 직접 실행에서는 질문당 약 1~2초 느린 것으로 관찰했다(질문당 1회). 그러나 아래 평가셋 재측정의 평균 지연은 켠 쪽이 오히려 짧아서 지연 영향은 일관되게 확인하지 못했다. 질문 원문 검색에서도 약한 질문에는 도움이 없고, 원문 검색 결과가 후보 자리를 일부 차지하므로 Router 결과의 3위 이하는 후보에서 빠진다. 판정 후보를 늘리면 해결되지만 토큰과 외부 전송량이 늘어서 택하지 않았다.
- 평가셋 결과: 아래 재측정에서 정답 평가셋 20문항의 1위 적중 수와 틀린 장면 수는 켜도 달라지지 않았다. 그래서 이 기능은 효과가 입증된 개선이 아니라 가능성을 확인한 단계로 남기고 기본값은 꺼 둔다.

이중 첫 검색 평가셋 재측정 **(2026-10-22)**

같은 정답 평가셋 20문항, 같은 모델(OpenAI), 같은 명령에서 이중 첫 검색만 끈 실행(v3)과 켠 실행(v4)을 각각 1회 다시 측정했다. 두 실행 모두 20문항을 끝까지 실행했고 오류는 0건이다. 현재 main 코드로 둘 다 새로 실행했으므로 3.2절의 v3 수치와는 별도의 실행이다.

표 **14.** 이중 첫 검색 끔 대 켬 **(OpenAI,** 같은 평가셋**,** 질문당 **1**회 실행**)**

지표	끔 **(v3)**	켬 **(v4)**	변화
Agent 1위 적중 수 (장면 질문 18개)	16	16	같음
Agent 상위 3개 적중 수 (18개)	16	16	같음
화자 질문 처리 성공 수 (4개)	3	3	같음
자막으로 알 수 없는 요청 거부 수 (2개)	2	2	같음
틀린 장면을 "찾음"으로 답한 수	1	1	같음 (같은 문항)
근거 없이 화자를 단정한 수	0	0	같음
Agent 결과 상태 (20개)	찾음 17, 못 찾음 1, 알 수 없는 요청 2	같음	같음
Agent 질문당 Tool 호출 (평균)	4.0 (총 80)	4.9 (총 98)	+1회/장면 질문
Agent LLM 호출 수 (20문항 전체)	42	42	같음
Agent 입력 / 출력 토큰	34,037 / 1,723	33,531 / 1,719	거의 같음
Agent 평균 / 최대 지연 (초)	2.56 / 6.24	2.38 / 6.00	켠 쪽이 오히려 짧음 (우연 범위)

- **1**위 적중 수에는 차이가 없다**.** 두 실행에서 Agent가 놓친 문항이 같다. 하나는 비슷한 말을 하는 장면이 여럿이라 다른 장면을 "찾음"으로 답한 문항이고, 다른 하나는 못 찾은 화자 질문이다. 이중 첫 검색은 이 두 문항을 구하지 못했다. 정답 평가셋에서는 이 기능의 정확도 이득을 확인하지 못했다.
- 비용은 예상대로 늘었다**.** 장면 질문 18개에서 검색(Tool 호출)이 1번씩 늘어 총 80회에서 98회가 됐다. LLM 호출과 토큰은 늘지 않았다. 평균 지연은 이 실행에서 오히려 짧게 나왔지만 0.2초 차이이고 질문당 1회 실행이므로 빨라졌다고 말할 수 없다.
- 실행마다 달라지는 부분이 있고**,** 이 비교는 기능 하나만 통제한 것이 아니다**.** temperature 0, seed 42로 같은 설정인데도 Router가 만든 영어 검색어가 5문항에서 두 실행 사이에 달랐다. 두 실행이 Router를 각각 새로 호출했으므로 C의 첫 검색 입력이 켬/끔 사이에서 같지 않았다. 따라서 이 결과는 "이중 첫 검색을 켠 실행과 끈 실행의 비교"이며 기능 효과만 분리한 것은 아니다. 같은 검색어를 재사용하는 A·B의 조건 ② 수치도 A 12 → 11, B 상위 3개 16 → 15로 움직였는데, 이중 첫 검색은 A·B에 적용되지 않으므로 이 차이는 Router 출력의 흔들림이다. Agent 결과가 바뀐 것은 반환 시점이 몇 초 달라진 2문항뿐이고 둘 다 정답 구간 안이었다. 향후에는 Router 검색어를 한 번 만들어 저장하고 두 실행이 그 검색어를 재사용하는 비교를 제안한다 (이번에는 수행하지 않았다).
- 결정: 정답 평가셋에서 이득이 없고 검색 비용만 늘었으므로 기본값은 꺼짐으로 유지한다. 이 평가셋 20문항 중 이중 검색이 필요한 "단어가 달라서 못 찾는" 문항은 이미 Router의 동의어 검색어로 대부분 찾고 있어서 보완 여지가 작았을 수 있다. 다만 이것은 가설이며, 마니또처럼 못 찾던 질문에서의 개선은 평가셋 밖의 직접 실행(위 표)에서만 확인했다. 효과 여부는 표본이 더 큰 평가셋이 있어야 판단할 수 있다.

4. 한계와 트레이드오프

4.1 Multi-Agent(Supervisor)로 나누지 않은 이유

- 설계 판단: 이 과제의 흐름(분류 → 검색 → 원본 대조 → 판정 → 재검색)은 단계 순서와 종료 조건을 코드가 정하고, LLM의 판단이 필요한 곳은 분류·판정·검색어 재작성 세 군데뿐이다. 각 단계가 독립된 목표를 가지고 스스로 다음 행동을 고르는 역할이 아니어서, 역할을 Agent로 나누고 Supervisor가 조율하게 할 실익이 제한적이라고 보았다. 나누면 "누구에게 맡길지"를 정하는 LLM 호출이 늘어 지연, 토큰, 실패 지점이 늘어날 것으로 예상했다. 도구도 3~4개뿐이고 호출 순서가 정해져 있어 지금은 코드가 MCP 도구를 정해진 순서로 호출한다(LLM이 도구를 고르는 function calling이 아니다).
- 비교는 하지 않았다: 단일 워크플로와 Multi-Agent를 같은 평가셋으로 비교하지 않았다. 위 판단은 구조에 대한 설계상의 이유이며 측정으로 확인한 결과가 아니다. 실제 실패 원인을 열어 본 것은 일부 문항에서 검색어의 단어 차이와 판정 기준이었다(4.2절).
- 서비스가 더 복잡해지면 전환한다: 공식 영상과 라이브 아카이브처럼 데이터와 검색 방식이 다른 콘텐츠를 함께 탐색하거나(라이브 자막을 확보해야 가능), "마니또로 자기 자신을 뽑은 뒤 누구 선물을 준비했는지 장면 순서대로 정리해 줘"처럼 여러 장면을 연결해 조사하는 서비스에서는 Search, Timeline, Verification Agent와 Supervisor로 역할을 나눌 이유가 생긴다. 구현하거나 측정하지 않았고, 도입한다면 같은 평가셋으로 정확도·지연·호출 수를 비교한다.

4.2 개선이 만든 약점과 정확도 한계

- 토큰 증가: v3는 호출 수가 줄었지만 프롬프트에 프로필과 규칙을 추가해 입력 토큰이 약 24% 늘었다(56,941 → 70,770). v3만의 비용은 따로 확인하지 못했고, LangSmith 합계 $0.30에서 v2 $0.1466을 뺀 약 $0.15로 추정한다(반올림 오차가 있다).
- **Agent**가 더 낫다는 증거가 약하다: Agent(16/18)와 B(15/18 또는 11/18)의 차이는 1~5문항이고 질문당 1회만 실행했다. 같은 설정(temperature 0, seed 42)으로 두 번 실행했을 때도 한 문항이 달랐다(17/18 → 16/18). 1문항 차이는 우연일 수 있다.
- 지연과 호출 증가: B는 평균 0.2초대인데 C는 OpenAI에서 평균 2.6초(최대 6.7초)이다. 질문당 LLM 호출이 약 2회이고 재검색이 있으면 4회로 늘어 토큰(비용)도 함께 늘어난다.
- 틀린 장면을 "찾음"으로 답함: 코드의 원본 대조는 cue가 실제로 있고 영상·시각·인용이 서로 맞는지만 확인한다. 그 장면이 사용자가 말한 장면인지는 확인하지 않는다. 그 판단은 LLM 의미 판정이 맡는데, 이 판정은 후보와 질문의 관련성을 보고 원본에 실제로 있는 다른 비슷한 장면도 정답으로 확정할 수 있다. v3에서 이런 사례가 1건 있었다("찾음" 17건 중 1건, 아래 실패 유형 점검 참고). 정답 구간 밖이면 오답으로 집계하며, 반환된 장면이 실제로 질문에 부합하는 라벨 한계 가능성은 확인하지 못했다.
- 번역 자막의 단어 차이와 결과의 변동: 질문의 단어와 번역된 자막의 단어가 다르면 후보에 오르지 않는다. 마니또 질문은 표현에 따라 찾기도 하고("자기 자신을 뽑은") 못 찾기도 했다("자기 자신 뽑은"). 이를 줄이려고 이중 첫 검색을 시도했다(3.4절).
- 동음이의어: "감"(과일/느낌)처럼 Router가 한 뜻만 골라 검색하면 다른 뜻의 장면이 후보에 오르지 않는다. 여러 뜻을 모두 검색하게 하는 방식은 gpt-4o-mini가 따라 주지 않아 폐기했다. Router를 gpt-4o로 바꾸면 해결되는 사례를 확인했지만 토큰 비용 때문에 기본값은 gpt-4o-mini로 두었다(평가셋 전체에 미치는 영향은 측정하지 못했다).



그림 6. 후속 질문의 실패 사례 (이중 첫 검색을 끈 채 실행 중이던 서버). 대상 없는 "다른 장면도 있어?"에 료칸과 무관한 캠핑 영상의 "bread" 장면을 정답으로 냈다(이 서버에서 같은 입력으로 6/6번 재현). 주제를 말한 후속 질문은 정상이다.

- 후속 질문: 위 사례처럼 "다른 장면도?"처럼 지시어만 있는 후속 질문에서 엉뚱한 장면이 나왔다. 다만 같은 입력이 최신 코드(이중 첫 검색 적용)로 띄운 서버에서는 3/3번 모두 료칸 관련 장면(48:11)이 나와 재현되지 않았다. 이중 첫 검색은 후속 질문에는 적용하지 않는 설계라 이 차이의 원인은 알지 못하며, 좋아졌다고 주장하지 않는다.
- 코드 리뷰에서 "Router가 다시 쓴 질문이 검증 기준까지 바꾸면 틀린 재작성이 스스로 정답으로 확정되는 문제"를 지적받아, 검증 기준은 원래 질문과 직전 대화로 고정하고 다시 쓴 질문은 검색에만 쓰도록 나눴다. 다만 이 규칙은 프롬프트로 지시하는 것이지 코드로 강제하는 검사가 아니다. 후속 질문의 정확도는 평가셋으로 측정하지 않았다.
- 이미 보여 준 장면을 제외할 때 모든 후보의 원문을 조회하던 방식을 고쳐, 제외한 뒤 남은 최대 3개만 조회하도록 했다(조회 횟수가 결과 수에 비례).

실패 유형 점검 (2026-10-22)

못 찾거나 틀린 원인을 "후보 검색 실패", "대표 시점 선택 실패", "의미 판정 실패"로 나누어 보기 위해, 평가셋 장면 질문 18개를 LLM 없이 로컬 검색으로 다시 실행했다(첫 라운드 단일 검색, 후보 3개). 정답 구간 ±10초와 겹치는 청크의 순위와, 그 청크가 상위 3개일 때 고른 대표 cue가 정답 ±10초 안인지를 확인했다. 질문당 1회 실행했다.

표 **15.** 첫 검색 기준 실패 유형 (장면 질문 **18**개, 문항 수)

첫 검색어	정답 청크가 상위 **3**개 안	**4~20**위	상위 **20**개 밖	상위 **3**개 안인데 대표 **cue**가 정답 밖
질문 원문(B, 조건 ①)	13	4	1	0
Router 영어 검색어(조건 ②, 끔 실행의 첫 검색어)	16	2	0	0

- 후보 검색(순위) 실패가 관찰된 유일한 유형이다. 정답 청크가 상위 3개 밖이면 판정 LLM이 정답을 볼 수 없다. Agent(C)가 놓친 두 문항(틀린 장면을 "찾음"으로 응답한 1건, 못 찾은 화자 질문 1건)은 첫 검색 기준으로 정답 청크가 각각 10위, 18위였다. 후보를 3개로 제한한 설계가 이 두 문항에서는 판정 이전 단계의 한계로 작용했다. 상위 20개까지 넓히면 정답 청크가 포함되는 문항이 원문 17/18, Router 검색어 18/18이다.
- 대표 시점 선택 실패는 이 점검에서 **0**건이었다. 정답 청크가 상위 3개 안일 때 고른 cue는 모두 정답 ±10초 안이었다. 다만 ±10초 허용 범위가 넓은 편이고 18문항이라, 선택 정확도가 높다고 일반화할 수는 없다.
- 의미 판정 실패는 별도로 측정하지 못했다. 정답 후보가 최종 3개 안에 있는데 판정이 놓친 사례는 평가셋에서 없었다(C의 상위 3개 적중과 1위 적중이 모두 16). 틀린 장면을 "찾음"으로 응답한 1건은 정답 후보가 첫 라운드 후보에 없던 상태에서 비슷한 다른 후보를 승인한 경우이므로, 후보 검색 실패와 의미 판정의 승인이 함께 작용했다고 본다. 어느 쪽이 결정적인지는 이 점검으로 가를 수 없다. 재검색 라운드에서 후보가 바뀔 수 있다는 점도 이 점검에는 반영되지 않았다.
- 영상과 음성은 분석하지 않는다. 입력은 영어 번역 자막과 일부 영상 정보뿐이고 화자는 알 수 없다.
- 정답 라벨의 근거와 확인 범위: gold의 정답 구간은 어시스턴트가 원본 cue를 검토해 제안했고 사용자가 20문항을 승인한 것이다(영상 화면을 문항마다 직접 확인한 것이 아니다). 영상 화면과의 대조는 영상마다 **3**곳, 모두 **9**곳의 표본 확인으로, 자막 시각이 영상과 어긋나지 않는지 보는 시각 동기화 점검이다. 정답 구간 전수 검증이 아니다.
- 평가 표본이 작다(장면 18문항, 질문당 1회 실행). p95 지연, 반복 실행의 분산, 못 찾아야 하는 질문의 정확도(확정된 문항 없음), 사람이 판정한 의미 품질, 다른 LLM은 측정하지 못했다. 개선은 평가셋의 실패 사례를 본 뒤 한 것이라 결과가 낙관적일 수 있고, 별도의 확인용 질문으로 검증하지 못했다.
- 실행 순서가 고정이라 앞선 실행이 만든 캐시가 뒤 조건의 지연을 줄였을 수 있다. 청크 길이는 10초/5초, 30초/15초, 40초/20초와 비교했으나(부록 E) 한국어 원문 조건의 검색(B)만 보았고 Agent(C)는 설정마다 다시 측정하지 않았다. RRF 상수(60)는 다른 값과 비교하지 않았다.
- 키워드 검색(BM25)은 공백으로 단어를 나누므로 일본어·중국어처럼 띄어쓰기가 없는 언어에서는 사실상 동작하지 않는다(영어 외 언어는 측정하지 않았다). 자막 형식은 SRT만 지원하고 한 영상의 다국어 자막은 다루지 않는다.
- 첫 기동: 검색 인덱스를 처음 만드는 데 CPU에서 약 200초가 걸려 MCP 초기화 제한(180초)을 넘기므로 미리 만들어 두어야 했다. Windows에서는 MCP 자식 프로세스가 가끔 시작에 실패했고, UTF-8 설정을 넘기고 오류 출력을 파일로 돌려 대응했다(원인을 완전히 밝히지는 못했다).
- **LangSmith**에서 **MCP** 도구 호출이 보이지 않는다: 그래프 단계와 LLM 호출만 올라가고 MCP 도구 호출은 따로 잡히지 않는다(개요 화면의 Tools 패널이 "No tool calls detected"). 도구 호출 수는 로컬 로그의 tool_calls로만 확인할 수 있다.
- OpenAI를 쓰면 질문과 후보 자막 일부(최대 3구간, 앞뒤 20초)가 외부로 전송된다. LangSmith도 같은 내용을 올리므로 비공개 프로젝트에서만 쓴다.

5. 느낀점과 이후 개선 포인트

5.1 느낀점

- 같은 질문인데 답이 나올 때도, 안 나올 때도 있었다. 마니또 장면은 표현을 조금 바꾸면 찾기도 하고 못 찾기도 했다. 같은 설정(temperature 0, seed 42)으로 두 번 실행해도 Agent가 만든 영어 검색어가 5문항에서 달랐다. 그래서 한 번 잘 나온 결과로 "된다"고 말할 수 없었고, 결과를 볼 때마다 같은 조건에서 다시 나오는지부터 의심하게 됐다.

- 정확도가 기대보다 낮았고, 원인을 전부 밝히지는 못했다. 검증과 재검색을 하는 Agent를 붙이면 많이 오를 줄 알았지만 Agent는 장면 질문 18개 중 16개를 맞혔고 일반 검색(Hybrid)과의 차이는 1~5문항이었다. 이중 첫 검색을 켜도 16에서 16으로 같았다. 못 찾은 질문 하나는 Router의 영어 검색어("manitto")와 번역 자막의 단어("Secret Santa")가 달랐다는 원인을 확인했지만, 나머지 실패(비슷한 장면 중 다른 장면을 고른 경우, 못 찾은 화자 질문)는 원인을 가리지 못했다.
- 다시 한다면 세 가지를 바꾸겠다. 첫째, 한 번에 여러 개를 고치지 않고 하나씩 고치고 하나씩 측정한다. v3에서는 네 가지를 한꺼번에 고쳐서 어느 변경이 효과를 냈는지 나누지 못했다. 둘째, 질문당 여러 번 실행해서 결과의 흔들림을 함께 잰다. 셋째, 기획을 달리해 여러 장면을 연결해 조사하는 더 복잡한 서비스로 잡고, 그에 맞는 Multi-Agent(Supervisor) 구조를 시험해 본다. 지금처럼 장면 하나를 찾는 서비스에서는 나누는 이점이 크지 않다고 판단했다(4.1절).

5.2 이후 개선 포인트

표 **16.** 이후 개선 포인트

영역	개선 내용	확인 방법
구조	지금의 단순한 "장면 하나 찾기"에서는 Multi-Agent로 바꾸지 않는다. 여러 콘텐츠 유형(영상·라이브)을 함께 탐색하거나 여러 장면을 시간 순으로 연결해 조사하는 서비스로 넓힐 때 Supervisor 구조로 전환한다(4.1절).	전환할 때 같은 평가셋으로 정확도·지연·호출 수·토큰 비교
검색 품질	RRF 상수(60)를 바꿔 비교하고, 청크 길이는 Agent(C)까지 포함해 설정별로 다시 측정한다. 청크 길이를 B에서 비교한 결과는 부록 E에 있다(20초/10초 유지).	정답 평가셋에서 1위 적중 수 비교(설정별 평가)
단어 차이	이중 첫 검색은 정답 평가셋에서 이득이 없어 기본값 꺼짐으로 유지했다. 표본이 더 큰 평가셋으로 다시 확인, 못 찾았을 때만 "혹시 이런 뜻인가요?"라고 되묻기, Router 모델과 동의어 사전 개선.	못 찾은 질문의 감소, 틀린 장면을 "찾음"으로 답한 수가 늘지 않는지 확인
후속 질문	"다른 장면도 있어?"처럼 이어서 묻는 질문은 직전 질문을 보고 검색어를 다시 쓴다. 이때 다시 쓴 검색어가 원래 질문의 주제를 벗어나지 않았는지 지금은 프롬프트로만 당부한다. 이것을 코드로 직접 검사하게 하고, 후속 질문만 모은 평가 질문을 따로 만든다.	후속 질문에서 엉뚱한 장면이 나오는 비율
사용성	검색 중에도 새 대화를 시작할 수 있게 한다(진행 중인 검색 취소 또는 대기열). 같은 영상에서 영상별 시점을 더 많이 보여주는 확장도 고려한다.	검색 중 새 대화 가능 여부, 판정 후보를 늘렸을 때 토큰·지연 증가량
관측	LangSmith에 MCP 도구 호출도 기록한다. 화면용 조회(앞뒤 자막, 영상 정보)를 Agent용 도구와 분리하는 방안을 검토한다. 현재는 "자막 데이터는 MCP 서버만 읽는다"는 단일 소유자 원칙 때문에 API 서버도 MCP를 거치므로, 조회를 API로 옮기려면 API가 자막을 읽을 수 있는 별도의 읽기 전용 조회 계층이 필요하고 이 원칙을 바꿀지 먼저 정해야 한다.	LangSmith 트레이스에서 도구 호출 확인
평가	① 같은 질문을 여러 번 실행해 결과가 얼마나 흔들리는지 잰다(지금은 질문당 1회). ② 설정을 고칠 때 쓰는 질문과 최종 확인에 쓰는 질문을 나눈다(지금은 고친 뒤 같은 질문으로 다시 재서 점수가 실제보다 좋게 나올 수 있다). ③ 자막에 정답이 없어서 "못 찾음"이 맞는 질문을 만든다(지금은 없어서 이 정확도를 못 쟀다). ④ 비슷한 장면이 여러 개인 질문은 정답으로 인정할 장면을 모두 정해 둔다.	반복 실행 간 편차, 못 찾아야 하는 질문의 정확도
대상 확장	영상마다 자막 언어를 설정하고, 다른 대상의 프로필과 자막 형식(SRT 외)을 지원한다. 띄어쓰기 없는 언어를 위한 단어 분리를 넣는다.	다른 대상·언어에서의 소규모 평가셋

부록 A. 실행 화면 모음



그림 7. 결과 화면. 검증을 통과한 장면 카드(원본 자막 인용), 선택한 장면의 영상, 원본 앞뒤 자막(강조된 줄이 근거). 화자는 "확인할 수 없음"으로 안내한다.

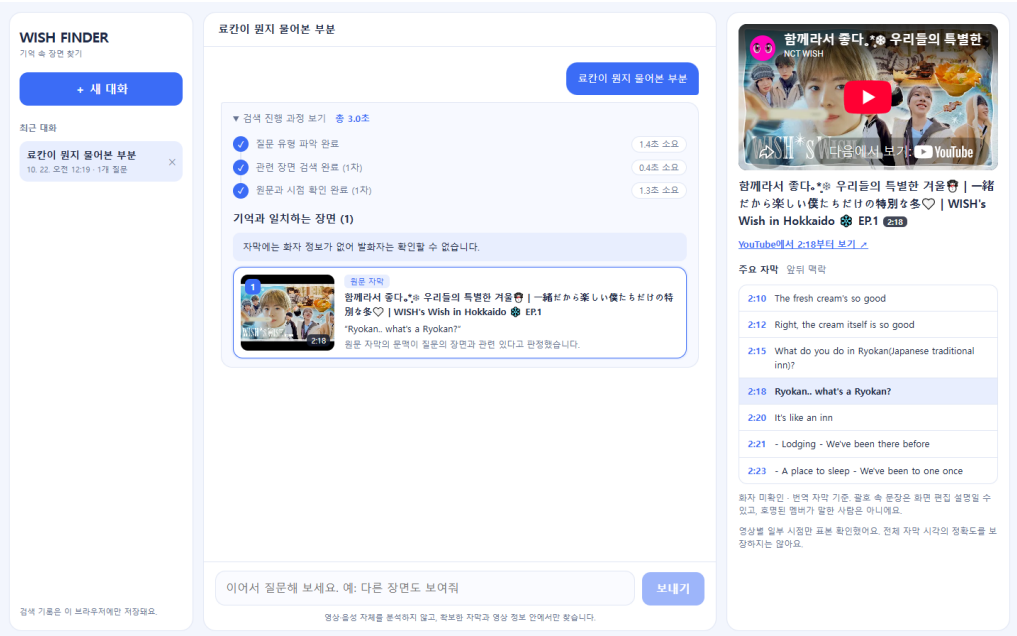


그림 8. Agent 진행 단계: 질문 유형 파악 1.4초 → 장면 검색 0.4초 → 원문·시점 확인 1.3초, 총 3.0초. 검증 전 결과는 화면에 나타나지 않는다.



그림 9. 화자 질문("곰이 물고기 물고 있다고 말한 사람이 누구야?"). 장면(58:04)은 찾되 "자막만으로 화자를 확인할 수 없습니다"라고 안내하고 말한 사람을 단정하지 않는다(총 5.0초).

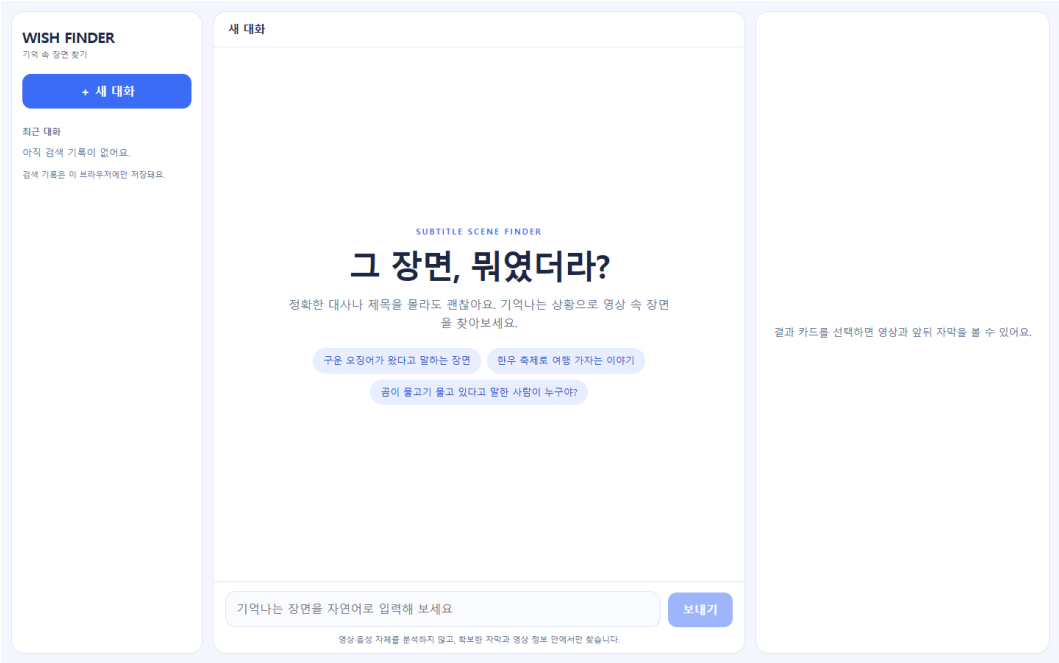


그림 10. 첫 화면. 예시 질문 3개가 있고, 검색 기록은 이 브라우저에만 저장된다.

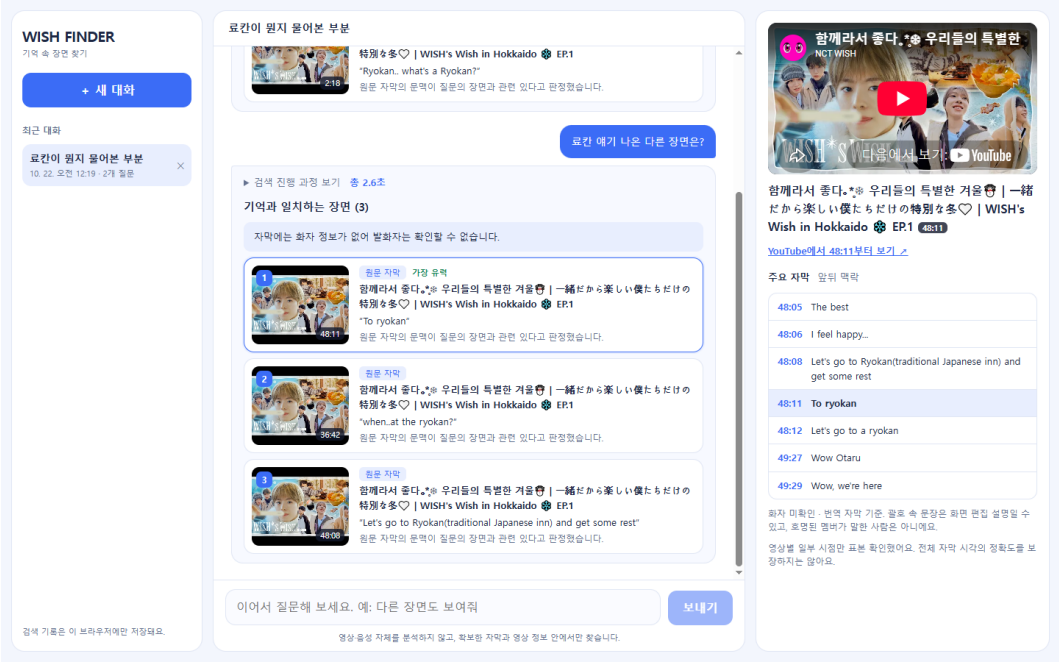


그림 11. 후속 질문("료칸 얘기 나온 다른 장면은?"). 직전 질문과 이미 보여 준 장면을 함께 보내 다른 장면을 찾는다. 서버는 대화를 저장하지 않는다.

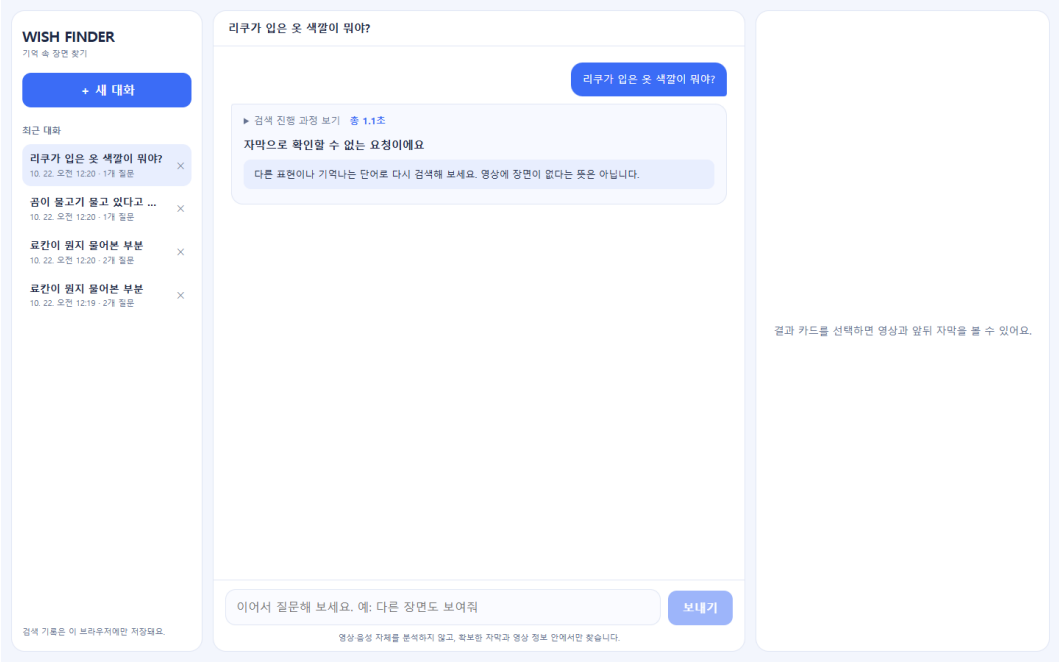


그림 12. 자막으로 알 수 없는 질문("…입은 옷 색깔이 뭐야?"). 검색 없이 1.1초 만에 "자막으로 확인할 수 없는 요청"으로 종료한다.

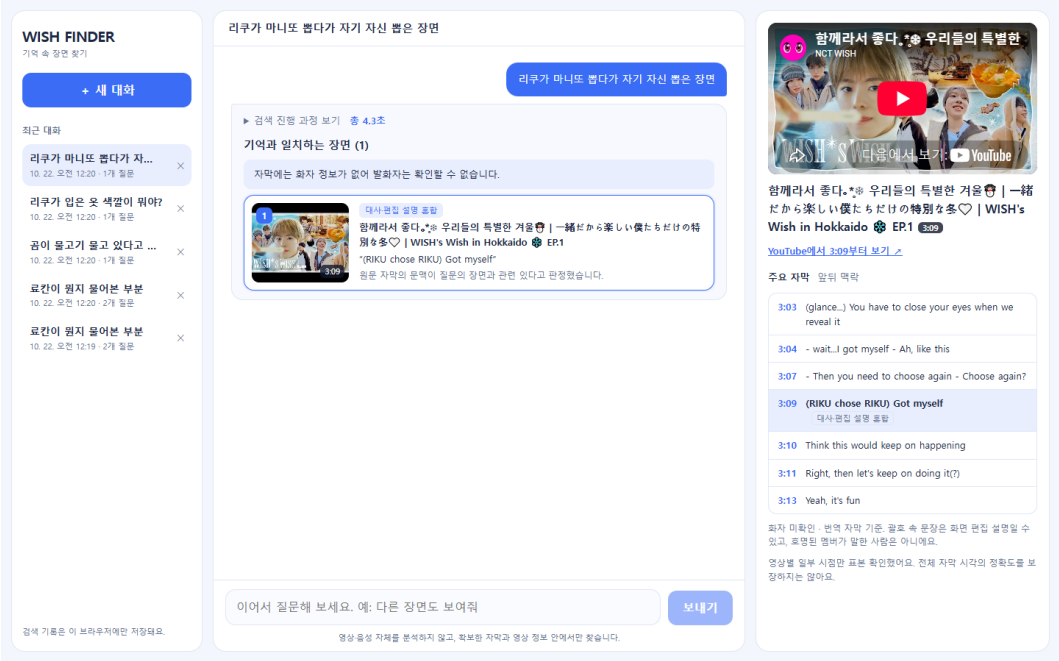


그림 13. 마니또 질문("…자기 자신 뽑은 장면"). 자막의 화면 편집 설명 "(RIKU chose RIKU)", 3:09 시점을 1라운드에 찾았다(총 4.3초, 이중 첫 검색 적용). 적용하지 않으면 못 찾았다(3.4절).

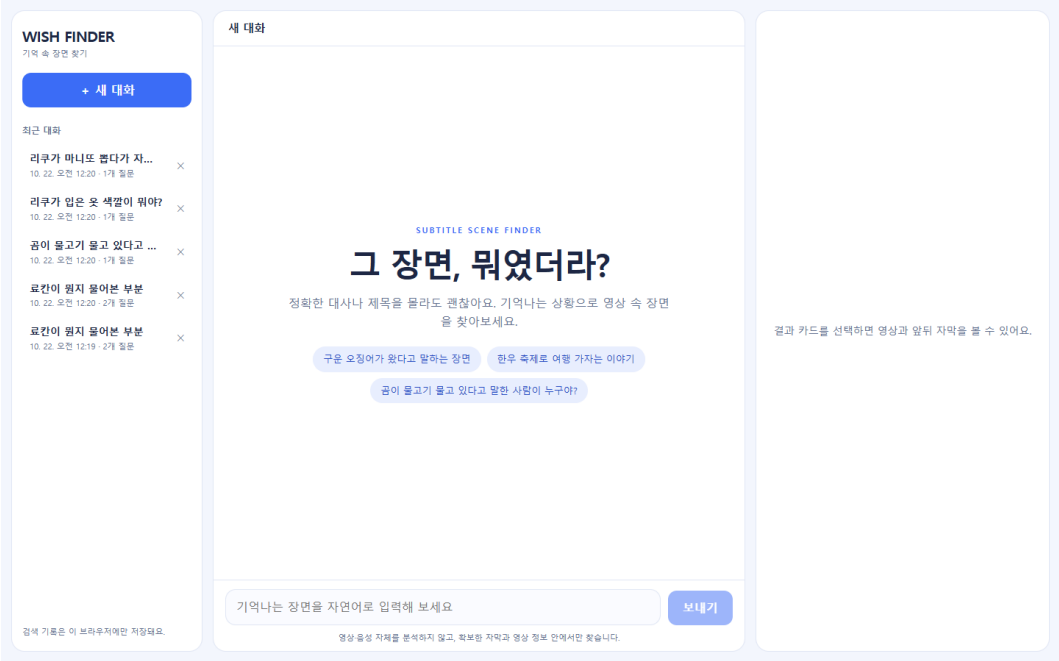


그림 14. 새로고침 후 검색 기록 복원. 저장된 결과는 다시 검색하지 않고 그대로 보여 준다.

부록 B. 실행·재현 방법

표 17. 실행·재현

항목	명령 / 설명
설치	py -3.11 -m venv .venv → .venv\Scripts\python.exe -m pip install -r backend\requirements.txt → npm --prefix frontend ci
백엔드 (OpenAI)	scripts/run_backend.sh openai (Windows는 Git Bash 또는 scripts\run_backend.ps1). .env에 OPENAI_API_KEY가 필요하고 과금 확인(y/N)이 나온다.
프런트엔드	scripts/run_frontend.sh → http://localhost:5173/ (127.0.0.1이 아닌 localhost로 접속)
테스트	.venv/Scripts/python.exe -m pytest backend/tests -q / npm --prefix frontend run test -- --run
평가	python -m eval.run_eval --dataset eval/private/gold.jsonl --systems A,B,C --control --max-llm-calls 150 --output eval/private/<결과>.json
공개 범위	원본 자막, 정답 평가셋, 질문별 결과, 검색 인덱스, .env는 Git에 올리지 않는다(eval/private/, data/raw/). 제삼자는 같은 자막 결과를 재현할 수 없고, 합성 자막 기반 테스트로 경로와 지표를 확인한다.

측정 환경과 재현 정보

표 **18.** 측정 환경과 재현 정보

항목	내용
측정 코드	2026-10-22 재측정(3.4절 재측정, 부록 D·E): main 커밋 a9a8169. v2·v3 실행(3.2·3.3절)은 결과 파일의 생성 시각(2026-10-21 15:13, 16:19 KST)만 있고 실행 당시 커밋은 기록하지 않았다.
실행 장비	재측정: Apple M5, 메모리 16GB, macOS 26.6.2, Python 3.11.15, CPU 추론(GPU 미사용). v2·v3 실행 장비는 결과 파일에 기록이 없다.
모델·라이브러리	LLM: gpt-4o-mini(분류·검색어 재작성), gpt-4o(판정), temperature 0, seed 42. 임베딩: BAAI/bge-m3(로컬 캐시만 사용, 다운로드 차단), 정규화 임베딩 간 내적(코사인 유사도)으로 유사도 계산, 배치 8. BM25: rank-bm25 0.2.2의 BM25Plus. sentence-transformers 6.1.0, LangGraph 1.2.16, MCP SDK 2.3.0, FastAPI 0.150.0.
시간 제한·재시도	요청 전체 60초, LLM 호출 1회 20초(초과 시 model_timeout), 재검색은 남은 시간이 25초 이상일 때만, 재검색 최대 2회. 모델·도구 호출의 자동 재시도는 없고 오류는 안전한 오류 코드나 부분 결과로 처리한다. 검색은 동시에 1건만 실행한다.
인덱스	청크별 임베딩을 미리 계산해 data/index/에 저장해 두고(처음 만들 때 CPU에서 약 200초) 서버 시작 때 불러온다. 청크 설정이 바뀌면 새로 만든다.
자막 데이터	NCT WISH 영상 3편, 영어 SRT, 모두 3,750 cue. 자막 마지막 cue 종료 시각 기준 길이와 cue 수: 캠핑 편 49분 25초·1,558 cue, 홋카이도 편 1시간 1분 23초·1,534 cue, 하우스 편 35분 23초·658 cue(영상 실제 길이와는 다를 수 있다). 사용자가 Wave YouTube Transcript 도구에 영상 URL을 입력해 SRT를 추출했다. 원본은 공개하지 않는다.
결과표와 실행 연결	3.2절 v2: day2-abc-gold-openai-v2.json, v3: day2-abc-gold-openai-v3.json. 3.3절 표: v3 실행. 3.4절 재측정과 부록 D: day3-dual-off.json, day3-dual-on.json. 4.2절 실패 유형 점검: 로컬 검색 점검(별도 결과 파일 없음). 부록 E: chunk-10-5.json, chunk-20-10.json, chunk-30-15.json, chunk-40-20.json. 모든 결과 파일은 비공개(eval/private/)이며, gold는 gold_sha256 d9819d48…로 승인 확인을 거친다.

부록 C. 설계 결정 기록 (ADR)

표 19. ADR 목록 (docs/adr/)

번호	주제
0001	자막 정보와 화자 미식별 처리
0002	Hybrid 검색과 다국어 질의
0003	LangGraph 역할 분리와 재검색 상한 (독립 Multi-Agent로 과장하지 않음)
0004	공식 MCP SDK 2.x MCPServer와 로컬 stdio 경계
0005	Vue + FastAPI POST SSE
0006	원본 cue 보존과 시간 창 근거 검증
0007	결과 카드의 YouTube 임베드와 오류 폴백
0008	LLM 백엔드 환경변수 교체 (Ollama 개발 / OpenAI 제출)
0009	대상 프로필, 번역 자막 프롬프트, 중복 재검색 제거
0010	대화형 3단 화면, 브라우저 검색 기록, 후속 질문
0011	첫 검색에서 Router 검색어와 질문 원문을 함께 검색 (이중 첫 검색, 기본 꺼짐)

부록 D. 정답 평가셋 (gold)

평가에 쓴 정답 평가셋 20문항의 구성은 3.1절과 같다(장면 질문 18: 일반 13, 같은 말을 하는 장면이 여러 개인 질문 1, 화자 질문 4 / 자막으로 알 수 없는 요청 2). 질문과 정답 시점은 평가셋을 공개하지 않기 위해 싣지 않았다.

표 **20.** 정답 평가셋 문항별 결과 **(OpenAI,** 질문당 **1**회 실행**)**

번호	유형	**B Hybrid** (원문)	**C Agent** (이중 검색 끔)	**C Agent** (이중 검색 켬)
1	직접 표현	오답	적중 (found)	적중 (found)
2	직접 표현	오답	적중 (found)	적중 (found)
3	바꿔 말한 표현	오답	적중 (found)	적중 (found)
4	바꿔 말한 표현	적중	적중 (found)	적중 (found)
5	직접 표현	적중	적중 (found)	적중 (found)
6	직접 표현	적중	적중 (found)	적중 (found)
7	바꿔 말한 표현	적중	적중 (found)	적중 (found)
8	바꿔 말한 표현	오답	적중 (found)	적중 (found)
9	바꿔 말한 표현	오답	오답 (found)	오답 (found)
10	화면 편집 설명	적중	적중 (found)	적중 (found)
11	화면 편집 설명	적중	적중 (found)	적중 (found)
12	화면 편집 설명	적중	적중 (found)	적중 (found)
13	바꿔 말한 표현	적중	적중 (found)	적중 (found)
14	비슷한 장면 여럿	적중	적중 (found)	적중 (found)
15	화자 질문	적중	화자 처리 성공 (found)	화자 처리 성공 (found)
16	화자 질문	적중	화자 처리 성공 (found)	화자 처리 성공 (found)
17	화자 질문	오답	화자 처리 실패 (not_found)	화자 처리 실패 (not_found)
18	화자 질문	오답	화자 처리 성공 (found)	화자 처리 성공 (found)
19	자막으로 알 수 없는 요청	해당 없음	거부 성공	거부 성공
20	자막으로 알 수 없는 요청	해당 없음	거부 성공	거부 성공

C는 이중 첫 검색을 끈 실행과 켠 실행의 결과가 문항별로 모두 같았다. B는 질문 원문을 그대로 검색한 값이다. 화자 질문 행의 "적중"은 관련 장면을 1위로 냈다는 뜻일 뿐 화자를 확인했다는 뜻이 아니다.

부록 **E.** 청크 길이 비교 **(2026-10-22)**

검색 청크는 영상별로 N초 창을 N/2초씩 이동해 만든다. 설계 때 정한 20초/10초가 맞는지 같은 정답 평가셋으로 비교했다. 청크 길이만 바꾸고 나머지는 같게 했고, LLM을 쓰지 않는 A(BM25)와 B(Hybrid)를 한국어 원문 조건으로 측정했다(Agent는 설정마다 다시 측정하지 않았다). 설정마다 검색 인덱스를 새로 만들어 비교했고, 질문당 1회 실행이다.

표 **21.** 청크 길이별 **B(Hybrid)** 비교 **(OpenAI** 없음**,** 질문당 **1**회**,** 한국어 원문 조건**)**

창 / 이동	청크 수	**B 1**위 적중 **(18**개**)**	**B** 상위 **3**개 적중 **(18**개**)**	**B** 평균 지연 **(**초**)**
10초 / 5초	1,712	11	13	0.19
20초 **/ 10**초 (기본값)	869	11	13	0.30
30초 / 15초	580	9	11	0.41
40초 / 20초	439	5	8	0.43

- 기본값**(20**초**/10**초**)**은 유지한다**.** 10초/5초와 점수가 같았고 청크 수는 절반이라 인덱스가 작다. 30초 이상으로 늘리면 적중 수가 떨어졌다(11 → 9 → 5). 청크가 길수록 한 청크에 여러 장면이 섞여 질문과의 거리가 흐려지고, 고른 근거 cue가 정답 시각에서 멀어지는 것이 원인일 수 있으나 이 실험으로 원인을 가르지는 못했다.
- 지연은 순서와 캐시에 영향을 받는다**.** 청크가 길어질수록 지연이 늘었지만 설정을 순서대로 한 번씩만 실행했으므로 지연 차이를 청크 길이의 효과라고 단정하지 않는다.
- 한계: 18문항, 설정당 1회라서 1~2문항 차이는 우연일 수 있다. 10초/5초와 20초/10초의 같은 점수도 "차이가 없다"는 뜻이지 "같다"는 증명은 아니다. 영어 검색어 조건과 Agent는 측정하지 않았다. 5초/2.5초처럼 더 짧은 창이나 더 큰 이동 폭은 시험하지 않았다.